

National University of Computer and Emerging Sciences



Assignment No. 1

Deep Learning for Perception

Fall 2026

Total Marks: 100

Submission Instructions

- Submit the complete **.ipynb file**, named **DL_ASS01_XXF_YYYY**, where XX represents the batch and YYYY represents the roll number.
- Submit a Word file containing the **GitHub repository link** and a one-page results summary.
- This is a group assignment (**maximum 2 members** allowed).
- Any submission not following the submission instructions will not be evaluated.
- Every plot must have axis labels, a title and a legend. Every reported number must be reproducible from the submitted notebook.
- Set and report a random seed. Runs that cannot be reproduced lose marks.
- Submit your own work. Accept your current limitations. Grow from your mistakes.
- **Plagiarism** and **AI** is strictly prohibited and leads to a straight zero.

Question: Building, Breaking and Fixing a Neural Network

Objective: Build a feedforward neural network end to end, deliberately push it into overfitting, then repair it using the regularisation and tuning methods covered in class. You will measure the effect of every design choice instead of assuming it.

1. Environment Setup

- Platform: <https://www.kaggle.com/>
- Accelerator: GPU T4 x2
- Dataset: Fashion-MNIST, <https://www.kaggle.com/datasets/zalando-research/fashionmnist>

Load the data, normalise pixel values to the range 0 to 1, and flatten each image into a 784-dimensional vector. Split the training data into a training set and a validation set using an 80/20 split. Keep the provided test set untouched until Part 7.

Report the number of samples in each split and the class distribution.

2. Part 1: Backpropagation From Scratch (15 marks)

Task: Implement a two-layer MLP in NumPy only. No autograd and no deep learning framework.

- Architecture: 784 to 64 hidden units with ReLU, then 10 output units with softmax.
- Loss: categorical cross-entropy.
- Implement the forward pass, the backward pass and a manual gradient descent update.
- Train on a subset of 5000 samples for 20 epochs and plot the training loss curve.
- Verify your gradients. Rebuild the same network in PyTorch, run one forward and one backward pass on the same batch with the same initial weights, then print the maximum absolute difference between your NumPy gradients and the framework gradients for each weight matrix.

State the maximum difference you obtained and comment on whether your implementation is correct.

3. Part 2: Baseline Model and Activation Study (15 marks)

Task: Rebuild the network in PyTorch or Keras with at least two hidden layers.

- Train one version each with sigmoid, tanh, ReLU and leaky ReLU in the hidden layers. Keep everything else fixed.
- Plot all four validation loss curves on a single figure.
- Record the mean absolute gradient of the first hidden layer at epoch 1 and at the final epoch, for the sigmoid model and the ReLU model.
- For the ReLU model, report the percentage of hidden units that output zero for every sample in a validation batch.

Explain what your gradient numbers show about vanishing gradients, and what the dead-unit percentage shows about ReLU. Two sentences each.

4. Part 3: Loss Functions (10 marks)

Task: Show that the loss must match the task.

- Train the classifier with categorical cross-entropy. Record test accuracy.
- Train the identical network with mean squared error on the one-hot targets. Record test accuracy.
- Plot both training curves together.
- Separately, select a tabular regression dataset, train a small MLP on it, and report MSE, RMSE and MAE on the test set.

Comment on why cross-entropy converges faster than mean squared error for classification.

5. Part 4: Optimiser Comparison (10 marks)

Task: Train the same architecture with four optimisers: plain SGD, SGD with momentum, RMSProp and Adam. Use the same learning rate for all four, then repeat with a learning rate tuned per optimiser.

Produce the following table.

Optimiser	Learning rate	Epochs to reach 85% validation accuracy	Final validation accuracy	Wall-clock time
SGD				
SGD + momentum				
RMSProp				
Adam				

Plot all four training loss curves on one figure. State which optimiser you would choose and why, in two sentences.

6. Part 5: Forcing Overfitting (10 marks)

Task: Deliberately create a large generalisation gap.

- Reduce the training set to 2000 samples and increase the network to at least four hidden layers of 512 units.
- Train for enough epochs that training accuracy exceeds 99%.
- Plot training and validation loss on one figure and mark the epoch at which the two curves separate.

- Report the generalisation gap as the difference between training and validation accuracy.

State whether this model has high bias, high variance, or both, and justify your answer using your own numbers.

7. Part 6: Regularisation Study (25 marks)

Task: Starting from the overfitted model in Part 5, apply each method below one at a time, changing nothing else.

- L2 weight decay, tested at a minimum of three values of lambda.
- L1 penalty. Report the percentage of weights below $1e-3$ after training.
- Dropout, tested at a minimum of three dropout rates.
- Batch normalisation.
- Early stopping. State the patience value and the epoch at which training stopped.
- Data augmentation using random horizontal flip and small random rotation.
- More training data. Retrain with 10,000 and then 20,000 samples.

Present all results in a single table of the following form.

Method	Setting	Train accuracy	Validation accuracy	Gap
Baseline (no regularisation)				
L2 weight decay				
L1 penalty				
Dropout				
Batch normalisation				
Early stopping				
Data augmentation				
More training data				

Then plot the generalisation gap against the strength parameter for L2 and for dropout, so that the effect of over-regularising is visible.

Write no more than 150 words comparing which method gave the largest reduction in the gap for the smallest loss in training accuracy.

8. Part 7: Hyperparameter Tuning with k-Fold Cross-Validation (15 marks)

Task: Select the final model properly.

- Define a search space over at least three hyperparameters, for example learning rate, hidden layer width and dropout rate.
- Run random search with at least 12 configurations, scored by 5-fold cross-validation on the training data.
- Report the search space, the mean and standard deviation of the cross-validation score for the top five configurations, and the selected configuration.
- Retrain the selected configuration on the full training set, combined with the best regularisation choices from Part 6.

- Evaluate once on the held-out test set. Report accuracy, macro precision, macro recall, macro F1 and a confusion matrix.

State the improvement over your Part 2 baseline in percentage points. If the tuned model did not improve, say so and explain why. An honest negative result scores full marks. A fabricated improvement scores zero.

Deliverables

- One notebook containing all seven parts, executed with visible outputs.
- All required plots, correctly labelled.
- The two summary tables from Part 4 and Part 6.
- A one-page summary in the Word file stating your final test score, the configuration that produced it, and the single change that helped most.
- GitHub repository link containing the notebook and a README with instructions to reproduce the result.

Marking Rubric

Component	Marks
Part 1: Backpropagation from scratch and gradient check	15
Part 2: Baseline and activation study	15
Part 3: Loss functions	10
Part 4: Optimiser comparison	10
Part 5: Forced overfitting and diagnosis	10
Part 6: Regularisation study	25
Part 7: Cross-validation and hyperparameter tuning	15
Total	100

Marks are deducted for unlabelled plots, missing random seeds, test-set leakage during model selection, and conclusions that are not supported by the reported numbers.